Blob Storage

什麼是 Blob Storage？

每次在系統設計面試裡提到「圖片存 S3」、「影片放物件儲存」，本質上你談的就是 Blob Storage。但真正理解它的運作方式，才能在面試中說清楚為什麼這是正確的選擇，以及它的限制在哪裡。

Blob（Binary Large Object）指的是任何大型的二進位資料——圖片、影片、音訊、PDF、備份、機器學習模型、日誌壓縮檔。Blob Storage（又稱物件儲存，Object Storage）是專門為儲存這類資料設計的系統。代表產品是 AWS S3、Google Cloud Storage（GCS）、Azure Blob Storage。

理解 Blob Storage 的關鍵，在於把它和你更熟悉的兩種儲存方式區分開來：

檔案系統（File Storage） 是你的筆電和大多數作業系統使用的方式。資料以樹狀的目錄結構組織，你用路徑（`/home/user/documents/report.pdf`）找到檔案。NFS、SMB、Amazon EFS 都是這類。它支援就地修改——你可以只改一個文件的某幾行，不需要重寫整個檔案。

區塊儲存（Block Storage） 是資料庫和虛擬機器底層用的那個。儲存空間被切成固定大小的區塊（block），作業系統透過低層次的位址存取它們。你不用操心「這個資料夾在哪裡」，只管「給我第 42 號到 58 號區塊」。AWS EBS、Google Persistent Disk 都是這類。它的存取延遲很低，適合需要隨機讀寫的場景。

物件儲存（Blob Storage） 走了完全不同的路。資料以「物件」為單位儲存，每個物件有一個唯一的 key，你透過這個 key 存取整個物件。物件是不可變的——你不能就地修改一部分，只能上傳一個新版本替換掉舊的。沒有資料夾的概念，key 看起來像路徑（`users/123/avatar.jpg`），但那只是 key 的命名慣例，底層沒有真正的目錄結構。

這個「不可變、以 key 存取整個物件」的設計，讓物件儲存可以做到一般檔案系統做不到的事：極高的耐久性、幾乎無限的水平擴展、全球複製，以及低廉的儲存成本。代價是你放棄了就地修改的能力，也失去了低延遲的隨機存取。

核心概念

Bucket 和 Object

bulbmat.org

物件儲存的最頂層容器叫 **Bucket**（AWS S3、GCS 的術語；Azure 叫 Container）。一個 Bucket 屬於某個帳號，有一個在全球唯一的名稱，並且關聯到一個特定的地理區域（Region）。實際的資料不會跨 Region 自動複製，除非你明確設定。

Object 是儲存的基本單位，由三個部分組成：

- **Key**：物件的唯一識別碼，在 Bucket 內唯一。看起來像 `uploads/2026/03/user-123/photo.jpg`，但實際上就是一個扁平的字串，沒有真正的目錄層級。
- **Data**：實際的二進位內容，從幾 bytes 到幾 TB 都可以。
- **Metadata**：描述物件的鍵值對，例如 `Content-Type: image/jpeg`、`Cache-Control: max-age=86400`。也可以附上自訂的 metadata，例如 `x-uploaded-by: user-123`。

```
Bucket: my-app-media
└── uploads/
    └── 2026/03/
        ├── user-123/photo.jpg     (Key, Data, Metadata)
        ├── user-456/video.mp4
        └── user-789/document.pdf
```

這看起來像目錄結構，但實際上這三個 key 是完全獨立的、扁平的條目。AWS S3 Management Console 只是把有相同前綴（prefix）的 key 顯示成「資料夾」而已。

耐久性和可用性

物件儲存最令人印象深刻的特性是耐久性。AWS S3 Standard 的耐久性是 **99.999999999%**（十一個九）。這意味著如果你存了 1,000 萬個物件，在一年內預期損失的物件不到一個。

這是怎麼做到的？底層靠的是跨多個可用區（Availability Zone）的冗餘儲存。一個 Region 通常有三個 AZ，每個 AZ 是一個獨立的資料中心，有各自獨立的電力和網路。S3 至少把資料跨三個 AZ 複製，其中一個 AZ 全毀也不會損失任何資料。

耐久性（Durability） 和**可用性（Availability）** 是兩件不同的事：

- **耐久性**：資料不會消失（十一個九）
- **可用性**：你能否存取到資料（S3 Standard 是 99.99%）

兩者都很高，但不完全相同。AWS 偶爾會有局部服務中斷，讓你暫時無法讀寫 S3，但你的資料不會因此消失。

最終一致性

buildneat.org

物件儲存的一致性模型值得理解，因為它在早期版本的 S3 上是一個有趣的限制（雖然 AWS 在 2020 年改善了這一點）。

現代 S3（2020 年後） 對 PUT 和 DELETE 操作提供強一致性（strong consistency）。你上傳一個物件後立刻讀取，保證讀到的是最新版本。

但在更廣義的分散式物件儲存設計討論中，最終一致性（eventual consistency）仍然是一個重要概念：系統允許短暫的不一致，但保證最終所有副本都會一致。這是 CAP 定理中選擇 AP（可用性 + 分區容錯）而非 CP（一致性 + 分區容錯）的典型例子。

在面試中，如果談到全球複製或自建分散式儲存系統，這個取捨會再次出現。

儲存分層

物件儲存的成本模型是按存取頻率分層的。大多數雲端供應商提供多個儲存等級，讓你根據資料的「熱度」（存取頻率）決定存放位置。

儲存等級	適合場景	存取延遲	儲存成本	取回費用
Standard（熱儲存）	頻繁存取的資料	毫秒	最高	無
Infrequent Access（溫儲存）	每月存取幾次	毫秒	中	有
Glacier Instant Retrieval	每季存取一次	毫秒	低	較高
Glacier Flexible Retrieval	備份、幾小時內取回	分鐘到小時	很低	更高
Glacier Deep Archive	長期歸檔、極少存取	最長 12 小時	最低	最高

這裡有個重要的取捨要記住：**儲存越便宜，取回越貴**。Glacier Deep Archive 的儲存成本可能是 Standard 的十分之一，但如果你突然需要大量取回資料，費用可能遠超過你省下的儲存成本。

生命週期策略（Lifecycle Policy）

手動管理每個物件的儲存等級不現實。雲端供應商提供生命週期規則，讓你自動化這件事：

```
{
  "Rules": [
    {
```

buildmoat.org

```
      "ID": "archive-old-uploads",
      "Filter": { "Prefix": "uploads/" },
      "Transitions": [
        {
          "Days": 30,
          "StorageClass": "STANDARD_IA"
        },
        {
          "Days": 90,
          "StorageClass": "GLACIER"
        },
        {
          "Days": 365,
          "StorageClass": "DEEP_ARCHIVE"
        }
      ],
      "Expiration": { "Days": 2555 }
    }
  ]
}
```

這個規則說的是：`uploads/` 前綴下的物件，30 天後移到溫儲存，90 天後移到 Glacier，一年後移到深度歸檔，七年後直接刪除。

在系統設計面試中，當面試官問「這些日誌或用戶上傳的歷史資料怎麼處理？」，生命週期策略是你的答案。不需要任何應用程式層的干預，儲存成本會隨時間自動下降。

buildmoat.org

存取控制

物件儲存的存取控制有幾個層次，理解它們的差異很重要。

Bucket Policy 和 IAM

IAM Policy 控制的是「哪個用戶或服務角色可以對哪個 Bucket 做什麼操作」。這是基於身份的控制，管理的是你組織內部的存取權限：

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::my-app-media/uploads/*",
      "Principal": {
        "AWS": "arn:aws:iam::123456789:role/AppServerRole"
      }
    }
  ]
}
```

Bucket Policy 是基於資源的控制，直接附加在 Bucket 上，可以控制跨帳號的存取，也可以強制所有物件必須加密存放、或拒絕未加密的傳輸。

公開 vs 私有

物件預設是**私有**的——只有 Bucket 擁有者能存取。

有些場景需要完全公開的物件，例如靜態網站資源、公開的 Open Data。但對大多數應用來說，「公開 Bucket」是個安全風險——AWS 的資安事件報告裡，S3 公開 Bucket 導致的資料外洩佔了很大比例。

正確的做法是 Bucket 保持私有，透過 **Presigned URL** 授予有時限的存取。Presigned URL 是由你的後端簽名產生的臨時 URL，帶有到期時間，任何持有這個 URL 的人都能在有效期內存取該物件，期限過後自動失效。（更多關於 Presigned URL 的細節，見 Handling Large Blobs 講義。）

版本控制

Bucket 可以啟用版本控制（Versioning）。啟用後，每次上傳同一個 key 的物件，舊版本不會被覆蓋，而是保留下來，分配一個新的 Version ID。

```
users/123/avatar.jpg  (Version: v3, 最新)
users/123/avatar.jpg  (Version: v2)
users/123/avatar.jpg  (Version: v1)
```

這帶來幾個好處：

- **意外刪除的保護**：刪除操作只是加上一個「刪除標記」，舊版本仍然存在，可以還原
- **應用程式層的回滚**：你可以快速切換回上一個版本的資源，不需要重新上傳

buildnout.org

- **覆寫保護**：就算有 Bug 在程式裡覆蓋了重要物件，資料也不會永久消失

代價是儲存成本：所有歷史版本都要收費。通常會搭配生命週期規則，自動刪除超過 N 天的舊版本，或只保留最近 N 個版本。

版本控制在系統設計面試中的用途：當面試官問到「怎麼確保重要資料不會被誤刪？」或「怎麼支援回滾功能？」，啟用 S3 版本控制是個乾淨的答案。

跨區域複製

Blob Storage 預設只在單一 Region 內的多個 AZ 之間複製，不會自動跨 Region。如果你想要在全球多個地點都有資料副本，需要明確設定跨區域複製（Cross-Region Replication，CRR）。

為什麼需要 CRR？

災難恢復（Disaster Recovery）：如果整個 Region 出事（雖然機率極低，但不是零），有另一個 Region 的副本可以讓你繼續服務。

法規遵循（Compliance）：某些法規要求資料必須備份在地理上分離的位置，或者要求資料必須留在特定國家境內（通常是不能跨 Region 複製的反向需求）。

讀取延遲優化：讓亞洲用戶從亞洲的 Region 讀資料，讓歐洲用戶從歐洲的 Region 讀——減少跨洋的網路延遲。

CRR 是非同步的：物件上傳到主要 Region 後，在幾秒到幾分鐘內複製到目標 Region，不是即時同步。如果你的系統需要強一致的地理複製，CRR 不夠用，需要更複雜的架構設計。

Blob Storage 在系統架構中的角色

作為應用程式的非結構化資料層

最常見的模式是讓資料庫存 metadata、Blob Storage 存實際內容：

```
用戶上傳頭像
    ↓
API Server 把圖片存到 S3 (key: users/{user_id}/avatar.jpg)
API Server 把 metadata 寫進 DB (user_id, avatar_url, updated_at)
    ↓
讀取時：DB 回傳 avatar_url → 客戶端直接從 S3 / CDN 取圖片
```

buildnotes.org

資料庫負責「這個用戶有沒有頭像、URL 是什麼」這類結構化查詢；S3 負責「給我這個 URL 對應的圖片 bytes」這種大型二進位資料的高效儲存和傳輸。兩者各司其職。

作為資料管線的中繼站

在事件驅動的架構中，Blob Storage 常常扮演不同處理階段之間的緩衝區：

```
原始影片上傳 → S3 (raw/)
    ↓ (S3 Event Notification 觸發)
轉碼 Lambda → S3 (transcoded/)
    ↓ (另一個 Event Notification)
縮圖產生 Lambda → S3 (thumbnails/)
```

每個處理步驟把輸出寫到 S3，下一個步驟從 S3 讀取，步驟之間完全解耦。這讓每個步驟可以獨立失敗和重試，不影響整體管線。

作為靜態資源的來源

靜態網站資源（HTML、CSS、JS、圖片）放在 S3，透過 CDN（CloudFront、Cloudflare）分發——這是幾乎所有現代 Web 應用的標準架構。S3 負責持久化儲存，CDN 負責在邊緣節點快取和加速分發。

```
Deploy: 靜態資源上傳到 S3
        ↓ CloudFront 從 S3 拉取並快取到全球邊緣節點
User Request → 最近的 CDN 邊緣節點 (快取命中) → 回傳
                    ↓ (快取未命中)
                    S3 (origin) → CDN 快取並回傳
```

作為資料湖（Data Lake）的底層

大規模的資料分析場景（ETL、機器學習訓練資料、日誌分析）通常把原始資料存在 Blob Storage：

```
應用程式日誌 → Kinesis → S3 (logs/year=2024/month=01/day=15/
*.parquet)
                                ↓
                    Athena / Spark 直接查詢 S3 上的資料
```

S3 搭配 Parquet 格式和 Athena（直接對 S3 上的資料跑 SQL），讓你不需要把資料搬進傳統的資料倉儲，就能做大規模分析。這個模式叫做 **Data Lake**，物件儲存是它

buildmoat.org

的基礎。

什麼時候在面試裡用這些

凡是大型二進位資料，立刻提出

系統設計面試中，只要題目涉及用戶上傳的內容，你的第一個反應就應該是：「這個應該放 Blob Storage，不放資料庫。」

「用戶可以上傳個人頭像和貼文圖片。這些圖片存在 S3，資料庫只存 metadata——圖片的 key、大小、上傳時間、狀態。資料庫不應該碰任何二進位資料，那會讓查詢效能和備份時間都崩掉。」

這句話說出來，面試官知道你有基本的架構直覺。

搭配 CDN 一起說

提到靜態資源或用戶內容的分發時，Blob Storage 和 CDN 幾乎永遠是一起出現的：

「圖片存在 S3，前面掛 CloudFront CDN。用戶第一次存取某張圖片，CloudFront 從 S3 拉取並快取在最近的邊緣節點；之後的請求直接從邊緣節點取，延遲從幾百毫秒降到個位數毫秒。S3 本身也因此承受的流量大幅降低。」

討論成本優化時提生命週期策略

當面試官問「怎麼控制儲存成本？」或「這些歷史資料怎麼處理？」：

「我們對不同冷熱程度的資料分層。最近 30 天的上傳放 S3 Standard，方便快速存取；30 到 90 天的移到 S3 Infrequent Access，成本大約少一半；90 天以上的歸檔到 Glacier，幾乎只需要 Standard 的十分之一費用。這些透過 S3 生命週期規則自動執行，不需要任何應用程式層的干預。」

常見面試情境

社群媒體平台（Instagram / Twitter）：「用戶上傳的圖片和影片直接存到 S3（透過 Presigned URL，伺服器不碰實際 bytes）。S3 有三個儲存層：最近 30 天的貼文用 Standard、舊貼文用 Infrequent Access、超過兩年的歸檔到 Glacier。媒體透過 CloudFront 分發，邊緣快取讓全球用戶都能低延遲存取熱門內容。」

檔案備份服務（Dropbox / Google Drive）：「備份資料的存取模式天然偏冷——大部分檔案備份後可能幾個月都不會存取。我們用 S3 Intelligent-Tiering，讓 AWS 根據每個物件的實際存取頻率自動移動到最便宜的等級，不需要我們自己預測和管理。對明確是歷史備份的資料，直接用 Glacier。」

日誌和監控系統：「應用程式日誌串流到 Kinesis Firehose，每小時壓縮後寫成 Parquet 格式存到 S3。用 Athena 對這些 Parquet 檔案跑 SQL 查詢，不需要另外維護

buildmost.org

日誌資料庫。生命週期規則確保三個月以上的日誌自動移到 Glacier，一年以上的刪除。」

電商平台：「商品圖片是最頻繁存取的資源，放 S3 Standard 並用 CloudFront 全球分發。訂單的 PDF 收據上傳後幾週內偶爾被查看，之後幾乎不存取，適合 Infrequent Access。年度財務報表是合規要求的歸檔資料，直接放 Glacier Deep Archive，成本極低。」

常見的 Deep Dive 問題

「Blob Storage 和資料庫都能存資料，怎麼決定用哪個？」

這是最基本的判斷，但值得說清楚。

用**資料庫**的場景：

- 需要按多個欄位查詢（`WHERE user_id = 123 AND status = 'active'`）
- 需要 ACID 事務（一個操作要同時改多筆記錄）
- 資料是結構化的、有明確的 schema
- 資料單元比較小（幾 KB 以內）

用 **Blob Storage** 的場景：

- 大型二進位資料（圖片、影片、文件、模型檔案）
- 不需要按內容查詢，只需要按 key 取整個物件
- 需要非常高的耐久性和低廉的儲存成本
- 資料一旦寫入很少修改（不可變的特性沒有問題）

判斷的粗略標準：**超過幾 MB、且不需要 SQL 查詢的東西，放 Blob Storage。** 資料庫存 metadata（key、大小、狀態、所有者），Blob Storage 存實際內容。

「怎麼防止 S3 Bucket 被公開曝露？」

S3 資料外洩事件幾乎都來自配置錯誤的公開 Bucket。在設計中要說清楚幾個防線：

預設封鎖公開存取：S3 提供「封鎖所有公開存取」的設定，在 Bucket 層級和帳號層級都可以設。這是應該作為預設值的設定，只有確實需要公開的 Bucket（例如靜態網站）才例外啟用。

私有 Bucket + Presigned URL：所有用戶內容的 Bucket 都設為私有，用 Presigned URL 授予有時限的個別物件存取權，而不是把整個 Bucket 設為公開。

buildmoat.org

S3 Access Analyzer：AWS 的工具，持續掃描你的 Bucket 設定，標記任何意外公開的 Bucket，讓你在資料外洩之前發現問題。

最小權限 IAM：應用程式的 IAM Role 只給它需要的最小權限——如果它只需要上傳到特定前綴，就不要給它讀取整個 Bucket 的權限。

「全球各地的用戶都要存取同一份資料，怎麼設計？」

這個問題的答案取決於資料的讀寫模式。

讀多寫少的資料（靜態資源、媒體文件）：CDN 是最直接的解法。S3 作為資料來源，CloudFront 在全球幾百個邊緣節點快取，用戶從最近的節點取資料，延遲極低。這不需要跨 Region 複製 S3 的資料，因為 CDN 會幫你做快取。

需要跨 Region 讀寫的資料（多 Region 部署的應用）：考慮 S3 跨區域複製（CRR）。主 Region 的寫入非同步複製到其他 Region 的 Bucket。各 Region 的應用從自己 Region 的 Bucket 讀，讀延遲很低；寫入發生在主 Region，複製延遲通常幾秒以內。

要注意 CRR 是單向或雙向可設定的。如果兩個 Region 都可能有寫入（雙向複製），需要設計好衝突解決策略——同一個 key 在兩個 Region 幾乎同時被更新，誰贏？S3 的雙向複製有「最後寫入者勝出」的預設語義，但要確保這符合你的業務邏輯。

合規要求資料不能離境：這是 CRR 的反向需求。用 S3 Bucket Policy 拒絕任何跨 Region 複製，並明確指定資料只能存放在特定 Region。在面試中提到這點，展示你知道合規需求可能和效能優化的方向相反。

「怎麼在 Blob Storage 上實作搜尋功能？」

這是個常見的陷阱，像是物件儲存不支援按內容搜尋，你不能對 S3 裡的圖片執行 `SELECT * WHERE 內容包含貓`。

但現實場景常常需要某種形式的搜尋。解法是**把需要搜尋的 metadata 存在其他地方**：

資料庫搜尋：把物件的 metadata（標題、標籤、上傳者、大小、格式）存在關聯式資料庫，對 metadata 做 SQL 查詢，結果包含 S3 key，再用 key 取物件。

全文搜尋（Elasticsearch / OpenSearch）：如果你需要對文件內容做全文搜尋，上傳後用 Lambda 提取文字內容（PDF 解析、OCR 等），把提取的文字索引到 Elasticsearch，搜尋的是索引而不是 S3 本身。

ML 向量搜尋：對圖片做以圖搜圖，用 ML 模型把圖片轉成 embedding vector，存進向量資料庫（Pinecone、pgvector）。搜尋時把 query 圖片也轉成 vector，找最近鄰。S3 存的是實際圖片，向量資料庫存的是圖片的語義表示。

buildroot.org

原則是：S3 是儲存的地方，不是搜尋的地方。任何需要搜尋的屬性，都需要在適合搜尋的系統裡有對應的索引。

總結

Blob Storage 是現代系統設計的基礎建材之一，幾乎所有涉及用戶上傳內容、靜態資源、日誌歸檔或機器學習的系統都會用到它。

在面試中記住幾個核心原則：**二進位資料不進資料庫，資料庫存 metadata、Blob Storage 存實際內容**；根據存取頻率用生命週期策略分層儲存，控制成本；私有 Bucket 搭配 Presigned URL，不輕易開公開存取；CDN 放在 Blob Storage 前面，讓全球用戶都能低延遲存取。

Blob Storage 不是萬能的。它不支援就地修改、不支援按內容搜尋、不適合低延遲隨機存取的場景。知道它的限制，和知道它的優勢一樣重要。把它和資料庫、CDN、搜尋索引組合起來，才能建構出完整的資料層設計。

buildmoat.org